



COMPUTAÇÃO PARALELA

AULA 1





CONVERSA INICIAL

Ao longo desta aula, faremos uma introdução sobre computação paralela e revisaremos alguns conceitos importantes para entender a evolução dos computadores, o que tornou paralelismo um tópico tão relevante. Mais especificamente, abordaremos a Lei de Moore com uma análise sobre os limites físicos dos *chips*. Relembraremos a arquitetura de computador de Von Neumann, no qual se baseiam quase a totalidade dos computadores de uso geral hoje, bem como os principais gargalos dos processadores ao longo de sua evolução tecnológica e em que o paralelismo se encaixa nisso.

Por fim, entenderemos o mapeamento de *cache*, outro conceito importante que nos auxilia a entender como tirar máximo proveito desse componente presente mesmo nos mais simples processadores atualmente. Em linhas gerais, entender um pouco sobre arquitetura básica dos processadores nos ajudará a aprender como estes evoluíram para uma estrutura paralela de hoje e como melhor usufruir do potencial tecnológico dessas máquinas.

Ao final desta aula, esperamos atingir os seguintes objetivos, que serão avaliados ao longo da disciplina da forma indicada.

Quadro 1 – Objetivos da aula

Objetivos	Avaliação
1. Entendimento sobre os principais componentes que formam uma arquitetura computacional, segundo Von Neumann	Questionário e questões dissertativas
2. Entendimento sobre os principais gargalos que surgiram ao longo da evolução das arquiteturas computacionais e as soluções adotadas	Questionário e questões dissertativas
3. Entendimento sobre as estratégias de mapeamento da memória principal pela <i>cache</i> do processador	Questionário e questões dissertativas

Fonte: O autor.

TEMA 1 – INTRODUÇÃO À COMPUTAÇÃO PARALELA

Em linhas gerais, a computação paralela busca as melhores soluções para aumentar a capacidade de processamento de dados, dividindo um problema em diversos passos menores que são realizados simultaneamente por



diversos núcleos de processamento. Buscamos tanto as melhores arquiteturas físicas para organizar múltiplos processadores e núcleos de forma eficiente quanto os melhores algoritmos paralelos, que diminuem o tempo de resposta de diferentes categorias de problemas.

1.1 Motivação

Uma pergunta importante quanto à motivação da computação paralela é: “O que você faria com mil vezes mais poder de processamento?”. Engenheiros, matemáticos, estatísticos, profissionais da área da computação, entre outros, frequentemente realizam atividades que demandam grande quantidade de processamento de dados e computação matemática. Simulações, computação científica, modelos estatísticos, entre outras atividades, são alguns exemplos de problemas que tradicionalmente demandam imensa quantidade de processamento de dados realizados por esses profissionais. Por vezes, tais atividades levam horas ou mesmo dias para finalizar sua execução.

Geralmente, dependendo da natureza do problema que se deseja resolver, é possível dividir o processamento desse problema em múltiplas partes autônomas, para serem executadas em paralelo e reduzirem para uma fração o tempo exigido para solucioná-lo. O campo da programação paralela estuda formas de otimizar esse processo e busca descobrir soluções eficientes para paralelizar categorias de problemas que, em uma análise superficial, não parecem ser paralelizáveis. Essa estratégia de dividir o problema em múltiplas partes torna possível realizar múltiplas execuções de um mesmo programa no mesmo intervalo de tempo, viabilizando certas atividades que, de outra forma, seriam simplesmente impossíveis por exigirem resposta com tempo muito restrito, como algumas aplicações em tempo real.

1.2 Aplicações

Dentre as principais áreas de aplicações da computação paralela, podemos citar:

- computação em tempo real, que demanda pontualidade na entrega de uma informação, pois esta perde sua finalidade de forma rápida. Nem sempre requerem necessariamente alto poder de processamento e



podem ser encontradas aplicações desse tipo em diversas situações, como dispositivos embarcados, sistemas multimídia e de controle;

- entrada/saída assíncrona, por exemplo, um servidor *web* realizando o *download* de diversos arquivos, ou atendendo a múltiplas requisições ao mesmo tempo;
- balanceamento de carga, que garante a distribuição de atividades entre diversos recursos de forma a garantir eficiência e efetividade em um sistema complexo como um servidor;
- sistemas com tolerância de falha, que podem paralelizar uma mesma atividade de maneira redundante permitindo que, no caso de problemas em parte do sistema, a atividade em execução continue sem interrupções;
- aplicações científicas, que são o exemplo mais clássico de uso de processamento paralelo devido à sua demanda por intenso processamento de dados e de computação matemática, destacando-se simulações e modelagens matemáticas;
- processamento de imagens e visão computacional, que, desde a própria renderização de imagens e vídeos em jogos até sua edição em *softwares* comerciais, utilizam intensamente o processamento paralelo. Devido à natureza de imagens e vídeos quanto a poderem ser analisadas em pedaços menores de forma independente, sem nenhuma perda, na maioria das aplicações se beneficiam muito de múltiplos núcleos de processamento.

TEMA 2 – LEI DE MOORE

A computação é um campo do conhecimento que ainda está muito longe de atingir seu potencial pleno. Novas descobertas são constantes e fazem o poder de processamento e armazenamento das máquinas aumentarem sempre. Portanto, é importante entender um pouco da história da evolução dos computadores para clarear nossa visão tanto sobre o presente quanto sobre o futuro do que nos aguarda dentro desse campo.

2.1 Tubos de vácuo para transistores

Os primeiros computadores da década de 1940 possuíam dimensões do tamanho de salas inteiras e chegavam a pesar algumas toneladas. Ainda assim,



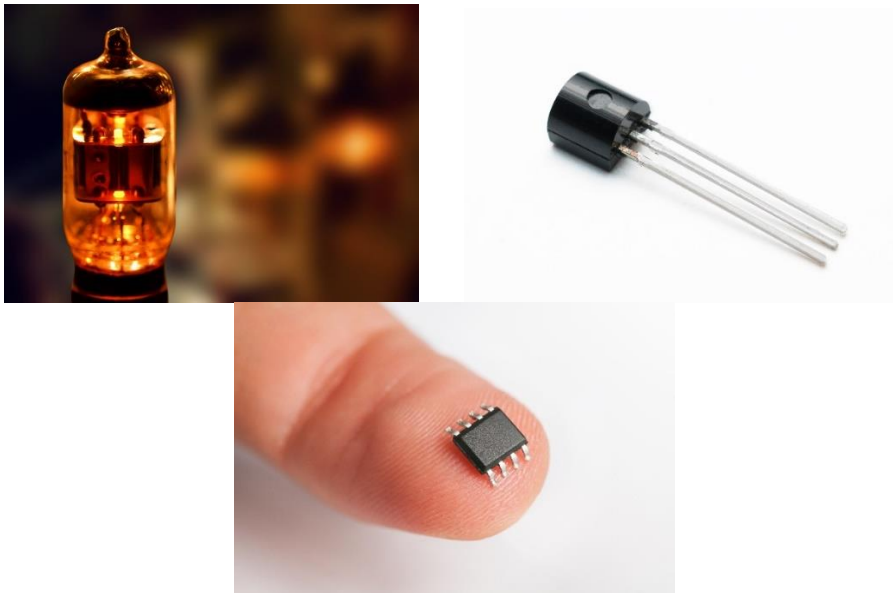
seu poder de processamento era uma pequena fração dos portáteis relógios eletrônicos, do inglês *smartwatch*, dos dias de hoje, que já contam com ao menos dois núcleos de processamento. Essa evolução vertiginosa do poder de processamento se deve muito à substituição dos tubos de vácuo, o componente básico que compunha os computadores de então.

Os tubos de vácuo em aparência se assemelhavam bastante a pequenas lâmpadas de alguns poucos centímetros; cada um possuía ao menos três eletrodos, o que lhes davam a possibilidade de intensificar um sinal elétrico ou cortá-lo completamente em uma corrente. Essa capacidade de ligar ou desligar um sinal tornou-o candidato perfeito para armazenar e processar um *bit* básico, valor **zero** quando desligado e valor **um** quando ligado, os “átomos” da computação. No entanto, esses tubos ocupavam significativo espaço físico, também apresentavam um elevado consumo de energia e, a cada poucas horas de funcionamento, ao menos um tubo necessitava ser trocado por decorrência de falhas.

Já no fim da década de 1940, essa tecnologia acabou sendo substituída por uma então recente invenção conhecida por transistor. Feito de semicondutores, possuía a mesma capacidade de amplificar e ligar/desligar sinais, ocupando um espaço físico muito reduzido, consumindo menos eletricidade, além de ser muito mais confiável e resistente a falhas. O primeiro computador baseado em transistores, chamado de Tradic, era aproximadamente quatro vezes mais barato, duas vezes mais rápido, além de ocupar um espaço muito reduzido em comparação a seus ancestrais baseados em tubos de vácuo.

Atualmente, os transistores evoluíram para sua próxima forma. Eles são agrupados em grande número, em formato miniaturizado, e são encontrados em praticamente todo o tipo de equipamento eletrônico; damos para esse agrupamento o nome de transistores de circuito integrado. Esse pequeno dispositivo de silício é também popularmente chamado, do inglês, de *chip*, *microchip* e até mesmo *nanochip*, dependendo das suas dimensões. O custo reduzido e seu desempenho nos colocou em uma nova era da eletrônica.

Figura 1 – Representação da diferença entre tubo vácuo de 1940, transistor de 1950, e circuito integrado composto de diversos transistores unidos em um mesmo componente dos dias de hoje



Créditos: EnsUPER/Shutterstock; Cristian Storto/Shutterstock; BigBlueStudio/Shutterstock.

2.2 Lei de Moore e os transistores

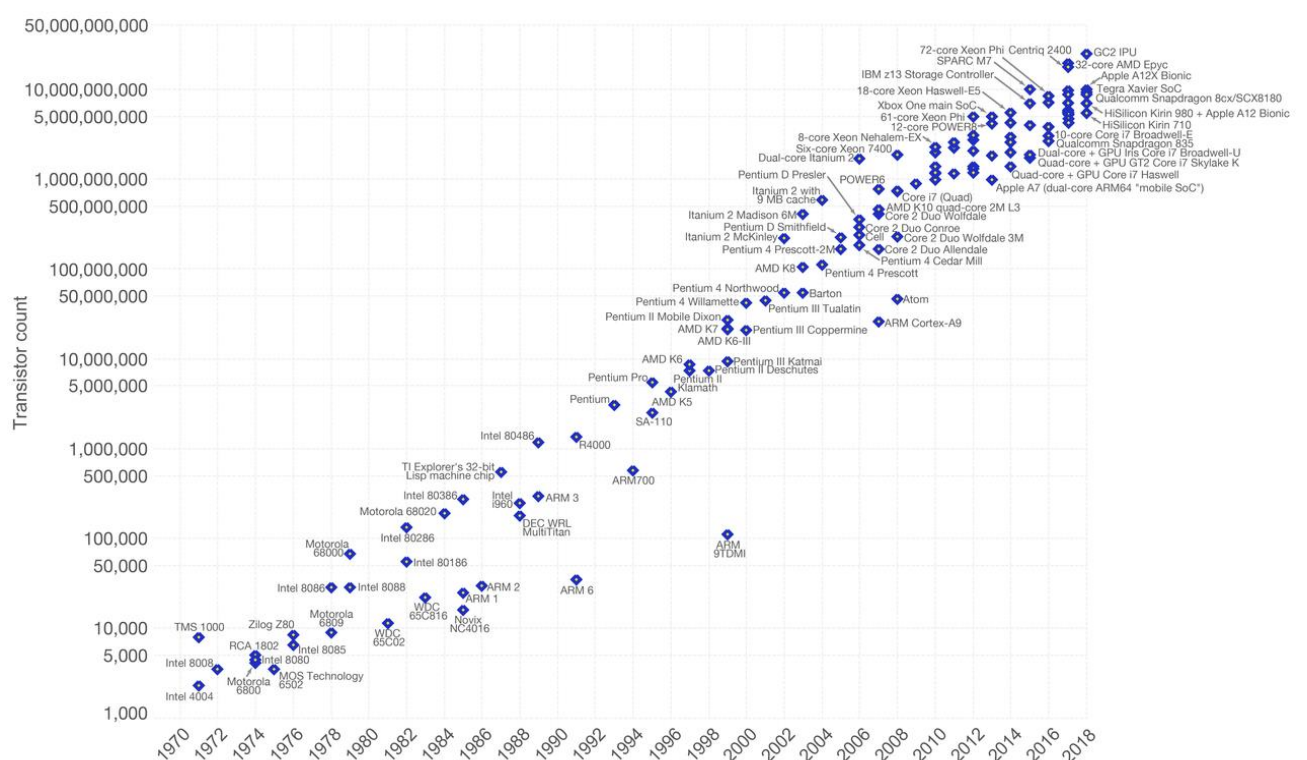
O cofundador da Intel, Gordon E. Moore, em meados dos anos 1960 profetizou que, a cada 18 meses, o número de transistores nos *chips* de processamento dobraria sem aumento no custo de produção e no consumo elétrico. Essa previsão ficou conhecida popularmente como “Lei de Moore”. Esse fenômeno realmente foi observado por muitas décadas, nas quais vimos um aumento constante e muito significativo no tempo de processamento dos algoritmos. No entanto, limitações de ordem técnica, como alimentação elétrica e capacidade de resfriamento, geraram limitantes para esse aumento constante e exponencial a cada nova geração do poder de processadores.

Damos o nome de *clock* do processador a um *microchip* capaz de regular o tempo no qual operam todas as funcionalidades dentro do computador. Esse *microchip* faz isso por meio de um cristal que vibra em uma frequência específica quando eletricidade lhe é aplicada. O intervalo de tempo entre as vibrações denota o tempo no qual um computador é capaz de realizar uma dentre as suas instruções mais rápidas. Esse intervalo de tempo é chamado de 1 *clock*. A velocidade dos computadores é medida por meio da velocidade desse *clock*. Por exemplo, 1MHz equivale a um milhão de ciclos ou vibrações por segundo, enquanto 2 GHz equivale a 2 bilhões de ciclos ou vibrações por segundo.

Os processadores atualmente não têm o mesmo aumento de *clock* por geração equivalente aos que ocorriam até ao começo dos anos 2000, visto que a redução no tamanho dos transistores dos circuitos também diminuiu as

distâncias internas aumentando, assim, sua velocidade. Porém, as fabricantes, para continuar atendendo à demanda por aumento no poder de processamento, desde então ampliam a quantidade de núcleos por processador. Além dos núcleos, também evoluiu a capacidade de cada núcleo executar múltiplas linhas encadeadas de execução, do inglês *threads*, dentre outras estratégias que lhes permitem continuar entregando cada vez mais processamento. Na Figura 2, vemos um gráfico que ilustra esse crescimento.

Figura 2 – Contagem de transistores dos anos 1970 até 2018, demonstrando a Lei de Moore



Fonte: Ourworldindata.org.

2.3 Escala de Dennard

Outra questão em jogo quando se fala da Lei de Moore é a chamada Escala de Dennard. Robert H. Dennard, um importante pesquisador que atuou na IBM, em um artigo de 1974, cunhou o princípio que leva seu nome. Esse princípio declara que a energia necessária para operar transistores em um dado volume físico é constante mesmo aumentando o número de transistores.

No entanto, temos encontrado limites para a Escala de Dennard, o que desacelerou a progressão da Lei de Moore. Os transistores se tornaram tão



pequenos que a Escala de Dennard não mais funciona como previsto e a energia necessária para operar o crescente número de transistores tem aumentado.

A questão da dispersão termal também é um fator importante no *design* de *chips* de computador. Bilhões de transistores ligando e desligando bilhões de vezes por segundo podem gerar uma quantidade muito grande de calor, a qual pode ser destrutiva para os delicados dispositivos de silício que operam com alta precisão e velocidade. O calor deve ser dissipado para algum lugar, e soluções de resfriamento são necessárias para manter a velocidade no processamento. Quanto mais transistores o sistema possuir, mais robusto deve ser o sistema de resfriamento para acomodar o aumento de temperatura. Computadores experimentais ficam próximo dos 9GHz de processamento utilizando resfriamento por nitrogênio líquido.

Saiba mais

Para mais informações sobre o *rank* dos computadores de maior processamento, acesse “CPU frequency: hall of fame”. Disponível em: <https://hwbot.org/benchmark/cpu_frequency/halloffame>.

O aumento do *clock* também implica um aumento na voltagem, o que acarreta um crescimento de ordem cúbica do consumo de energia do *chip*, o que aumenta novamente a temperatura, necessitando de ainda mais resfriamento, que, por sua vez, demanda ainda mais energia. No fim, a demanda elétrica e a geração de calor ultrapassam os ganhos pelo aumento do *clock*.

Existe também uma questão de ordem física para essa barreira. Uma das principais razões para o *clock* aumentar com a diminuição dos transistores reside no fato de que, juntamente com o transistor, diminui também o componente denominado *gate*, a parte do transistor que se move para ligar/desligar o próprio circuito. Com um *gate* menor, a distância percorrida também era menor, o que aumenta a velocidade; no entanto, os modelos mais recentes já possuem *gate* de aproximadamente a espessura de um átomo de silício, limitando em termos físicos a possibilidade de ainda maiores reduções.

2.4 Além da diminuição do *clock*

Uma vez tendo entendido a dificuldade de desenvolver *chips* ainda mais rápidos devido às limitações do aumento do *clock*, a melhor maneira encontrada para aumentar o poder de processamento dos sistemas computacionais tem sido



por meio de *design* de *chips* com múltiplos núcleos, expandindo em importância o estudo da computação paralela.

TEMA 3 – ARQUITETURA VON NEUMANN

Vamos retomar mais uma vez os primeiros anos da computação, na década de 1940, quando os primeiros computadores digitais foram desenvolvidos. Esses computadores eram chamados de Colossus (1943) na Inglaterra e Eniac (1945) nos Estados Unidos, ambos com finalidades militares, como quebra de criptografia inimiga. Esses computadores apresentavam uma série de limitações, sendo uma delas o fato de que seus dispositivos internos precisavam ser remontados sempre que se desejasse executar um programa diferente, processo que poderia tomar até mesmo dias.

O matemático Von Neumann, em meados do século XX, deixou importantes contribuições nas mais diversas áreas do conhecimento, como estatística, economia, física nuclear, teoria dos jogos, ciência da computação, hidrodinâmica, além de vários campos da matemática. A arquitetura de computadores que leva seu nome, arquitetura Von Neumann, foi uma dessas contribuições que modela, de forma geral, em um alto nível de abstração, os computadores que utilizamos até os dias de hoje.

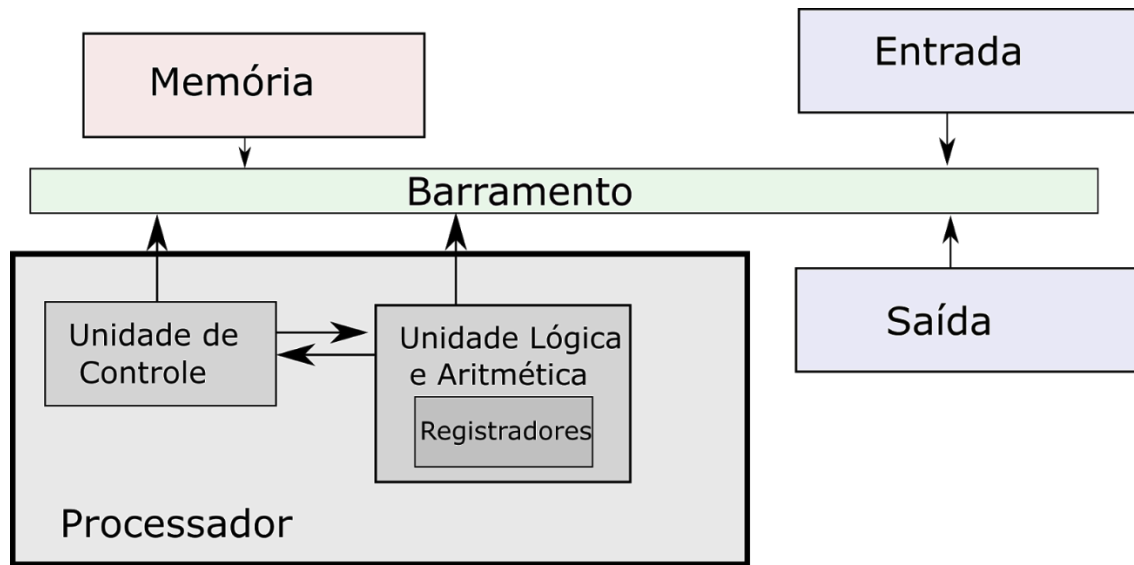
A arquitetura Von Neumann consiste em quatro elementos principais:

- a **memória**, que armazena dados e todas as instruções do programa;
- o **processador**, que executa instruções sobre os dados na memória em ciclos de leitura/execução/escrita;
- a **entrada/saída**, que representa demais dispositivos que interagem com o barramento, como teclado, monitor e afins;
- o **barramento**, que liga memória, processador e a entrada/saída do sistema.

Essa arquitetura, ao colocar o código do programa na memória, resolveu o problema da necessidade de remontar a máquina para cada novo programa. Agora, bastava simplesmente carregar um novo programa na memória e executar novamente. Na Figura 3, vemos um modelo dessa arquitetura.



Figura 3 – Arquitetura Von Neumann



3.1 Composição do processador

É importante entender como são compostos o processador e os demais componentes com os quais ele interage para discutirmos em detalhes sua evolução e como chegamos nas arquiteturas paralelas modernas. No Quadro 2, vemos uma descrição resumida de cada componente ilustrado na Figura 3. Seus nomes são mais frequentemente vistos na literatura em inglês.

Quadro 2 – Descrição dos principais componentes da arquitetura Von Neumann

Componente	Nome em inglês	Descrição
Unidade de Controle (UC)	<i>Control Unit</i>	Responsável por decodificar as instruções e guiar como os dados vão se mover pelo sistema.
Unidade Lógica Aritmética (ULA)	<i>Arithmetic Logic Unit</i>	Realiza os cálculos aritméticos e lógicos requisitados pelas instruções do programa.
Barramento	<i>BUS</i>	Cabeamento responsável por transportar os dados pelo sistema.
Memória de acesso aleatório (RAM)	<i>Random Access Memory</i>	Memória de acesso aleatório: memória de uso genérico que armazena os dados e o programa.
Entrada/Saída (E/S)	<i>Input/Output</i>	Demais dispositivos que recebem ou enviam novos dados ao sistema.
Registradores	<i>Registers</i>	Espaço de memória com propósitos específicos.



Como visto no Quadro 2, os registradores são espaços mínimos de memória com propósito muito específico, diferentemente da memória principal, que é um espaço amplo o qual serve para múltiplos propósitos. Cada registrador fica localizado dentro do processador e atende a uma demanda principalmente na gerência da atividade de acessar instruções e dados na memória principal. Os registradores acabam sendo a memória mais rápida que temos, por ficarem dentro dos próprios processadores e serem diretamente endereçados por conta de seu propósito específico. No Quadro 3, descrevemos alguns dos principais registradores de forma resumida.

Quadro 3 – Descrição dos principais componentes da arquitetura Von Neumann

Registrador	Nome em inglês	Descrição
Contador de Programas (PC)	<i>Program Counter (PC)</i>	Grava a posição na memória da próxima instrução que deve ser executada.
Registrador de Instrução (IR)	<i>Instruction Register</i>	Armazena a próxima instrução que será executada.
Registrador de Endereço (MAR)	<i>Memory Address Register</i>	Registra o endereço de memória que será acessado tanto para escrita quanto leitura.
Registrador de Dados (MBR)	<i>Memory Buffer Register</i>	Enquanto MAR registra o endereço, a MBR registra o conteúdo que será lido/escrito da memória.
Acumulador	<i>Accumulator</i>	Grava temporariamente o cálculo feito pela ULA.
Registrador da instrução corrente (CIR)	<i>Current Instruction Register</i>	Armazena o endereço de memória da instrução mais recentemente carregada da memória.

3.2 Arquitetura Harvard

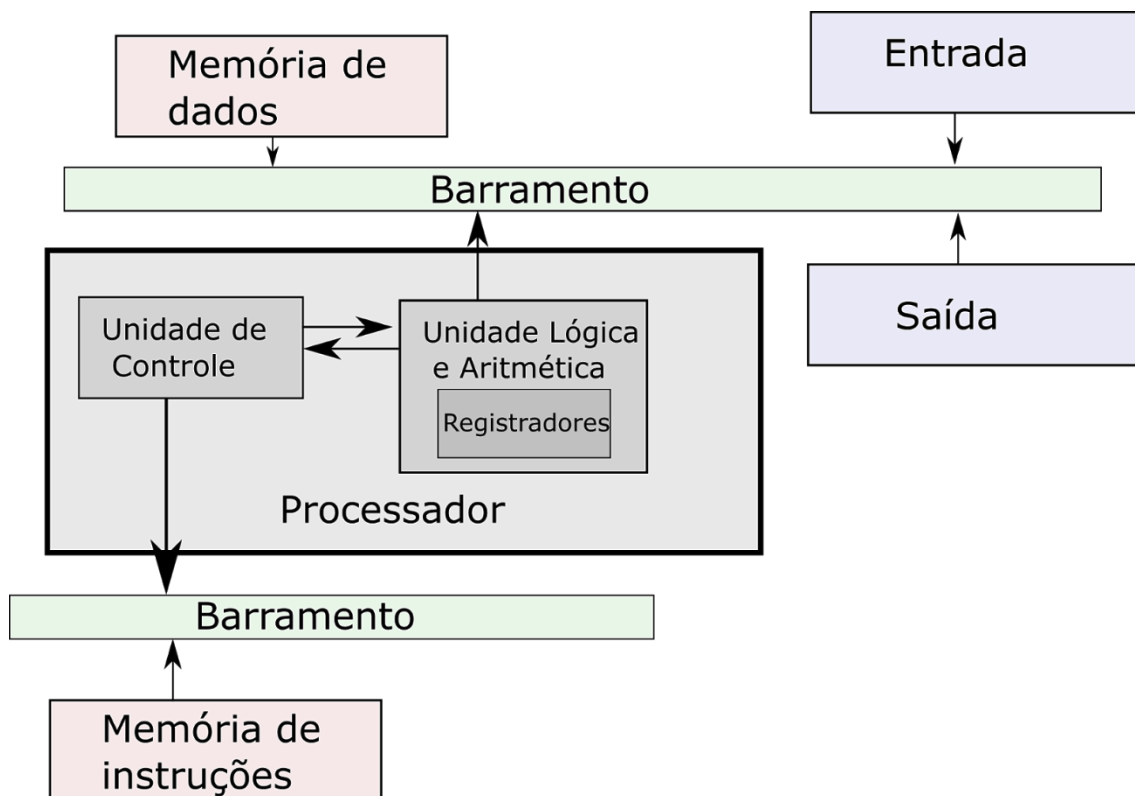
Paralelamente, na universidade de Harvard, foi desenvolvido outra arquitetura de computadores semelhante à de Von Neumann. A arquitetura Harvard também possui uma unidade de controle que se comunica com uma unidade lógica aritmética a qual é ligada na memória. Porém, a diferença está justamente na memória que é dividida em duas, a memória das instruções e a memória dos dados. Cada uma conta com seu próprio barramento, como é ilustrado na Figura 4.



A arquitetura Harvard é amplamente utilizada hoje em certos dispositivos embarcados como os chamados DSPs, do inglês *Digital Signal Processors*, que são monofunções e não necessitam de flexibilidade na execução de seus códigos. O programa é salvo na memória ROM, do inglês *Read Only Memory*, que permite reescrita, e utiliza a tradicional memória RAM para os dados.

Nos processadores dos computadores de uso geral modernos, tipo ARM e Intel, a arquitetura de Von Neumann é adotada, embora internamente as memórias *cache* mais próximas ao processador separem os dois tipos de memória. Mesmo assim, consideramos a arquitetura como sendo Von Neumann, pois possuem um barramento único para acessar a memória principal em um nível de hierarquia mais alto.

Figura 4 – Arquitetura Harvard



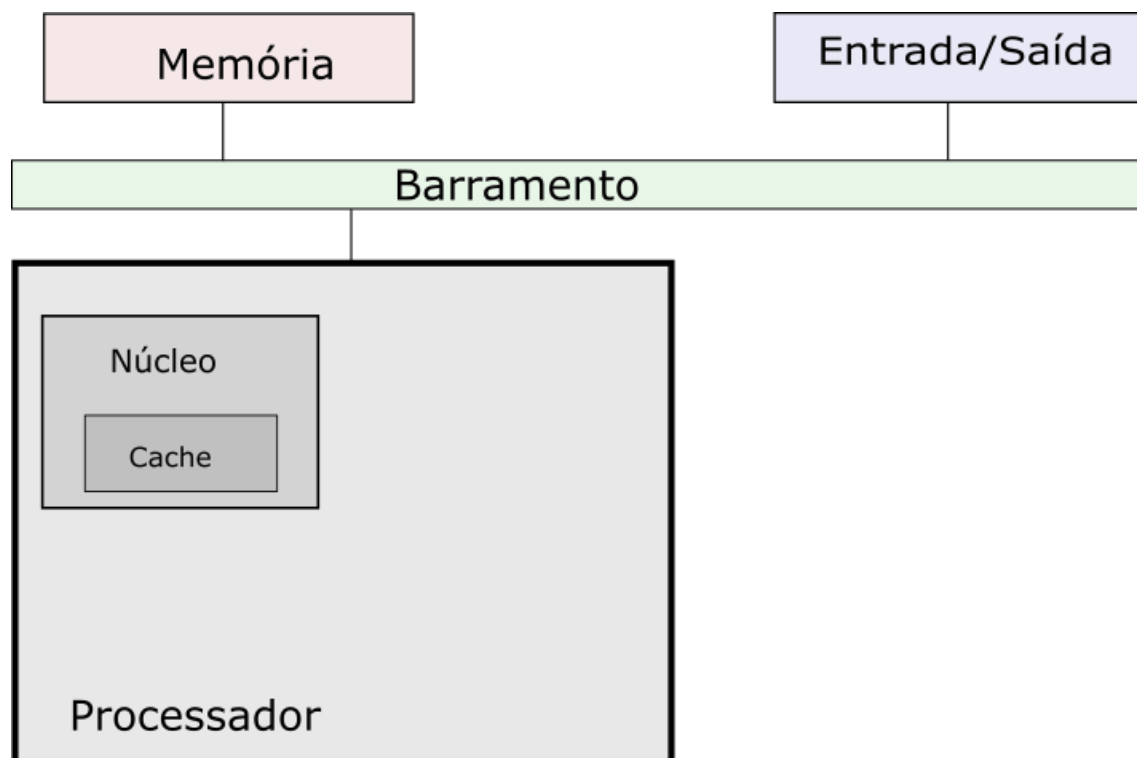
TEMA 4 – EVOLUÇÃO DOS PROCESSADORES E PRINCIPAIS GARGALOS

A arquitetura de computadores Von Neumann evoluiu ao longo dos anos e encontrou diversos “gargalos” para seu desempenho. Para melhor entender isso, vemos na Figura 5 que a unidade de processamento (CPU, do inglês *Central Processing Unit*) se comunica com a memória por meio do barramento de dados (no inglês, *data bus*). Como tanto os dados quanto as instruções do



programa passam pelo barramento, esse fluxo constante gera o primeiro gargalo dessa arquitetura. Cada instrução nova vai ocupar o barramento para buscar a próxima instrução ou dado, tornando-o muito requisitado e diminuindo o desempenho do sistema como um todo. O tempo para trazer uma informação da memória principal via barramento é muito superior ao tempo gasto pelo processador para executar uma instrução, deixando o processador ocioso e o barramento constantemente ocupado. A solução para esse primeiro gargalo, chamado de gargalo de código, foi a implantação de uma pequena memória interna no processador, conhecida como *cache*. Na imagem a seguir, dividimos a CPU em núcleo (*core*, no inglês) e *cache*.

Figura 5 – Modelo da arquitetura von Neumann com *cache*

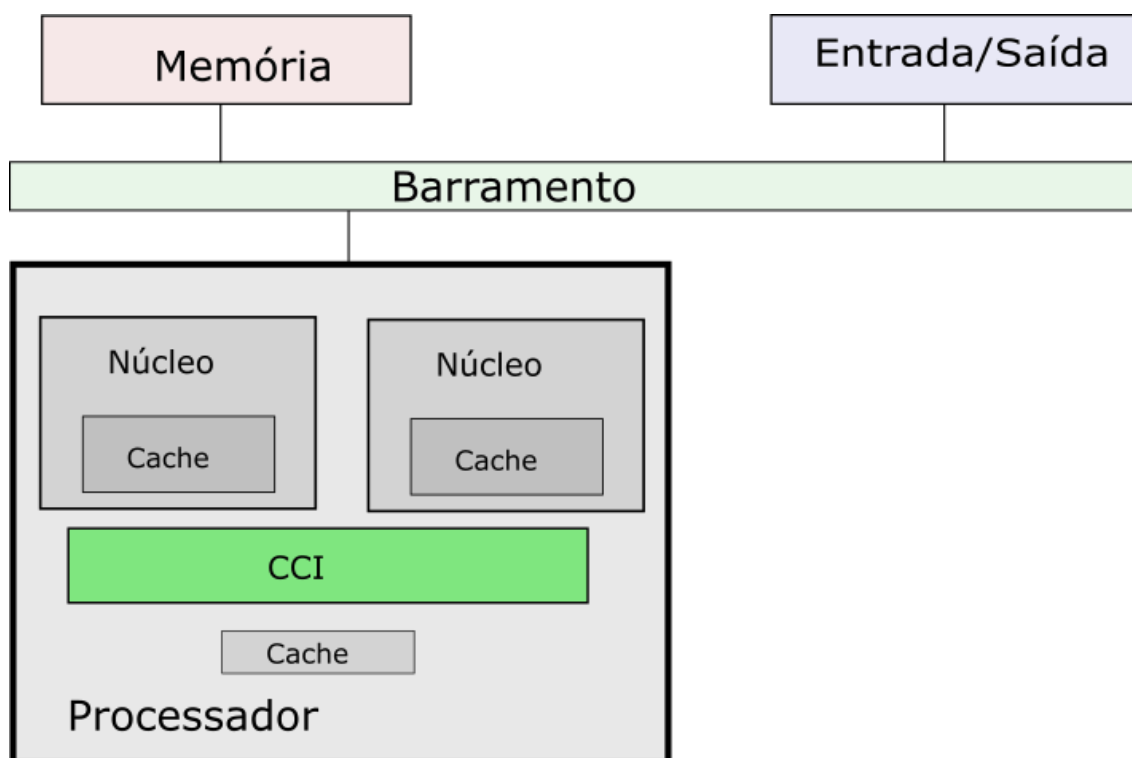


Essa arquitetura funcionou de forma muito eficiente sem grandes alterações entre as décadas 1970 e 1990. Nesses anos, os processadores aumentaram seu poder de processamento em uma taxa exponencial, no ritmo que propôs a “Lei de Moore”. No entanto, com a diminuição dos circuitos e o aumento do número de instruções executadas, a temperatura passou a ser outro problema e o incremento exponencial no *clock* dos processadores deixou de acontecer por volta dos 2GHz~4GHz atingidos em meado da década de 2000. Esse efeito gerou um segundo gargalo, que pode ser chamado de gargalo de núcleo. A solução proposta pela Intel foi implantar um segundo núcleo dentro do

processador, aproximadamente dobrando a velocidade de processamento para certas aplicações sem a necessidade de modificar o *clock*.

Ainda assim, isso gerou outro problema, visto que, como era de se esperar, cada núcleo possui sua própria *cache* para evitar o gargalo de código. Entretanto, se um dado que está na *cache* do primeiro núcleo tiver de ser acessado pelo segundo núcleo, temos um novo problema, pois o dado que reside na memória estará obsoleto, e o segundo núcleo precisará acessar a *cache* do primeiro núcleo. Isso pode ser resolvido por *software* que vai novamente ocupar o barramento, ou como é comercialmente feito, adicionando um *hardware* responsável por gerir a coerência desses dados nas *caches*, chamado de CCI (do inglês, *cache coherent interconnect*). Na Figura 6, vemos um modelo desse sistema.

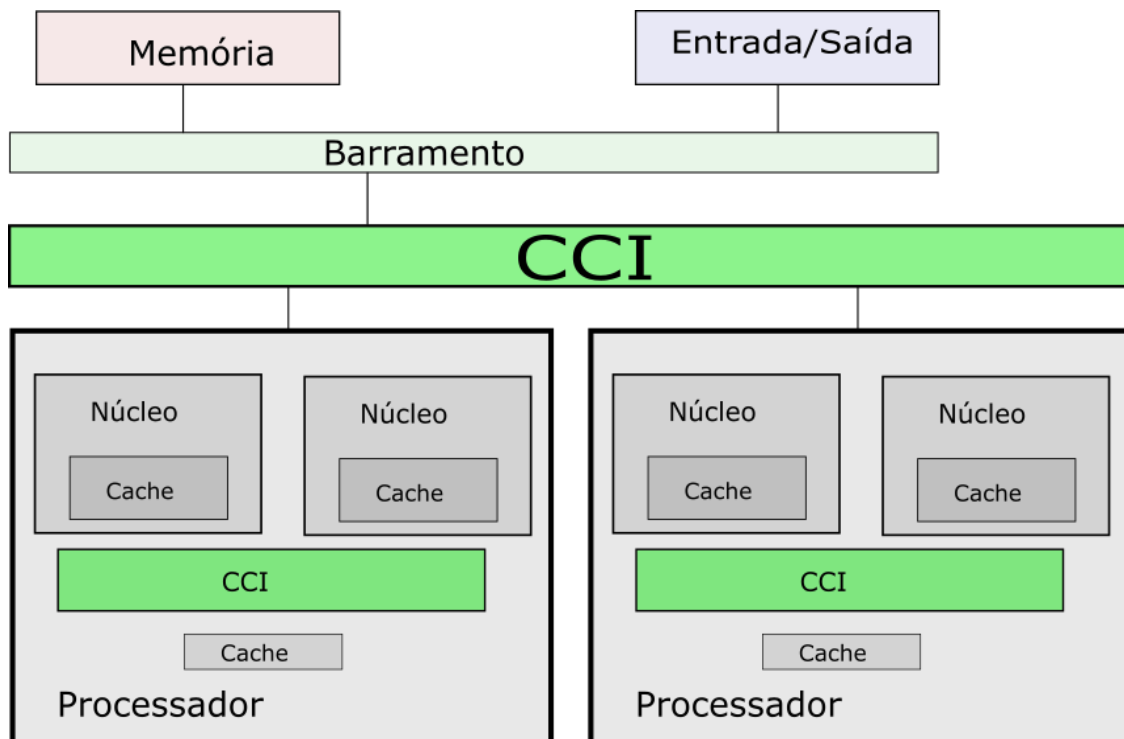
Figura 6 – Modelo da arquitetura von Neumann com *cache* e CCI



Essa nova arquitetura *multi-core* também atingiu um limite na sua capacidade de processamento de dados, gerando um terceiro gargalo, o gargalo de CPU. Como solução, foi adicionado ao sistema um segundo processador *multi-core*. Para evitar o problema com a coerência das *caches*, uma nova CCI foi adicionada para interligar os *chips* e não somente os núcleos. Na Figura 7, vemos esse novo modelo.



Figura 7 – Modelo da arquitetura von Neumann multiprocessada



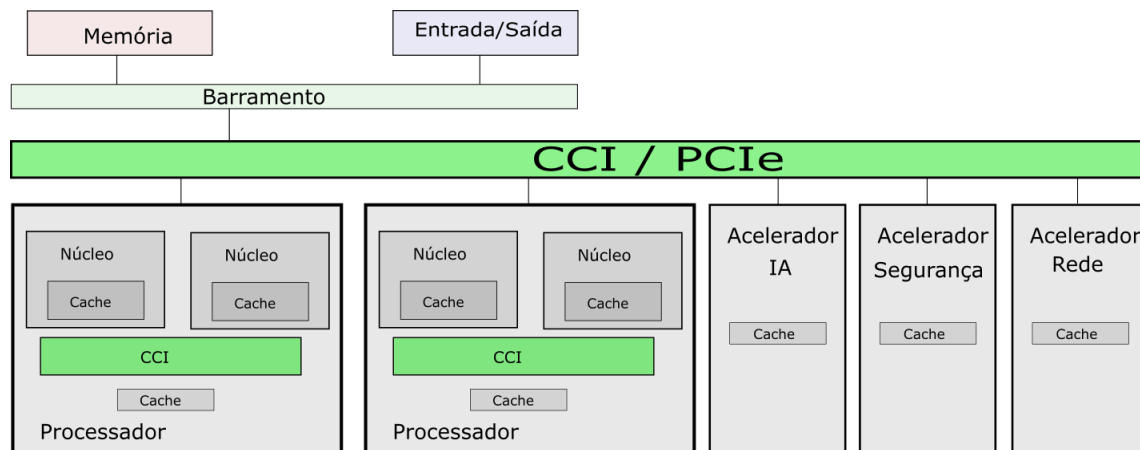
Nos últimos anos, vemos um grande aumento na relevância e na utilização do processamento de dados transmitidos pela internet. Dispositivos móveis, popularização do *streaming* de vídeo e ainda mais recentemente atividades ligadas à *deep learning* e à inteligência artificial. Esse aumento na demanda, especialmente sobre os servidores na nuvem, gerou um quarto gargalo, que chamamos de gargalo da nuvem. Algumas dessas atividades são tão específicas e recorrentes que, para otimizar o processamento, novos *hardwares* específicos, chamados de aceleradores, foram criados para lidar com elas. Por exemplo, aceleradores de IA, segurança, rede, entre outros. Cada acelerador de propósitos específicos também disputa pelo barramento e acesso aos dados, portanto, também ganharam suas próprias *caches*. Para facilitar a comunicação desses dispositivos com o restante do sistema, a CCI foi expandida para se comunicarem com as novas *caches* dos novos *chips*.

Por fim, chegamos ao mais recente gargalo. No caso dele, não se trata de uma questão de ordem arquitetural, mas sim uma questão prática. Cada companhia desenvolveu seu *kit* composto por todos esses elementos em licença prioritária, incluindo os CCIs, inviabilizando a combinação de *chips* desenvolvidos por outras companhias; chamamos esse quinto gargalo de gargalo das companhias. Para solucionar essa questão, uma organização formada por diversas das companhias que produzem processadores



desenvolveu um padrão aberto para que esses aceleradores pudessem se comunicar entre si, o chamado CCIX. O padrão roda sobre a interface PCI Express (PCIe), que já é amplamente utilizado em servidores I/O. Na Figura 8, vemos essa arquitetura agora composta pelos aceleradores, conectados via PCIe.

Figura 8 – Modelo da arquitetura von Neumann com aceleradores e conectados via PCIe



Saiba mais

Para mais informações sobre a história e evolução dos processadores e seus gargalos, acesse o artigo “Von Neuman Bottlenecks and CCIX”. Disponível em: https://community.cadence.com/cadence_blogs_8/b/on-the-beat/posts/von-neuman-bottlenecks-and-ccix.

TEMA 5 – MAPEAMENTO DE CACHE

A memória *cache* foi uma importante adição para as arquiteturas de processadores. Trata-se de uma memória cujo espaço é muito reduzido e, em compensação, é muito mais rápida quando comparada à memória principal. A *cache* possui conexão direta com o processador, faz um mapeamento dos acessos e copia certas regiões-chave da memória principal evitando que a CPU ocupe o barramento principal a cada acesso, de forma geral, desobstruindo o barramento para outros dispositivos. Essa estratégia diminui drasticamente o tempo de acesso aos dados.

De forma geral, as *caches* desejam tentar “adivinhar” quais serão os próximos endereços lidos e, desse jeito, minimizar as viagens para a memória principal. Chamamos de *acerto* quando o endereço buscado estava na *cache* e



de *falha* quando este não estava fazendo o processador buscar na memória principal via barramento.

Existem dois princípios importantes que baseiam algumas decisões de *design* das *caches*.

- O chamado princípio da **localidade espacial** diz que, quando um dado é acessado, seus vizinhos na memória tendem a ser acessados também. Pense na tarefa de lermos os dados de um vetor, uma imagem, uma *string*, entre outros. Todos esses dados são salvos de forma sequencial na memória. Então, ter uma *cache* que salve não somente os dados da posição atual, mas também das posições vizinhas em um bloco permite uma grande chance de acertar as próximas leituras; portanto, as *caches* comerciais operam dessa forma.
- O outro princípio é chamado de **localidade temporal**, que entende que um dado que foi recentemente acessado possui grande chance de ser acessado novamente, o que também é verdade. Depois de lermos os dados de uma variável/vetor, é comum acessarmos a mesma posição novamente para gravar uma informação nova, ou mesmo acessar várias vezes para realizar cálculos distintos. Por isso, todo o acesso na memória principal é prontamente salvo na memória *cache*.

A memória *cache* é completamente transparente para a CPU, o que significa que o processador entende que está acessando a memória principal. Os endereços que a CPU aponta não fazem referência à *cache*, ou seja, utilizamos o endereço da memória principal para buscar as informações na *cache*.

Se tivéssemos que procurar por um dado individualmente na *cache*, um por um, o tempo para essa operação seria superior ao de acessar a memória principal, inviabilizando o uso da *cache*. É importante que, com o endereço da memória principal, seja possível acessar de forma direta e imediata os dados. Para isso, existem diferentes estratégias de mapeamento.

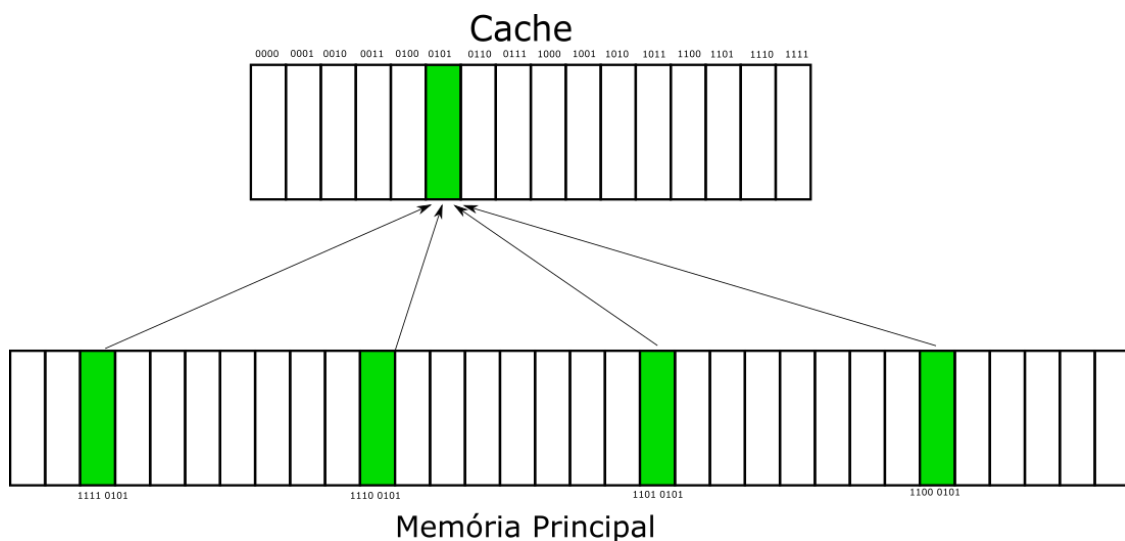
Por mapeamento entendemos a configuração da *cache* de forma que tenhamos uma espécie de mapa que indique quais registros foram copiados no *cache* e sua localização.



5.1 Mapeamento direto

Uma estratégia bastante natural é o chamado mapeamento direto, nele utilizamos os *bits* menos significativos de um endereço da memória principal para mapear a *cache*. Suponha que tenhamos uma memória principal de 256 posições (endereços de 8 *bits*), sendo mapeado para uma *cache* de 16 posições (endereços de 4 *bits*). Todos os endereços com os mesmos 4 *bits* finais serão mapeados para a mesma posição na memória. Por exemplo, os espaços de memória terminados com os *bits* de endereço 0101 da memória principal (1111 0101, 0010 0101, 0001 0101, entre outros) utilizarão o mesmo espaço de memória endereçado como 0101 na *cache*. Na Figura 9, vemos ilustrada essa situação descrita no exemplo.

Figura 9 – Mapeamento direto da memória *cache*: diversos endereços da memória principal mapeados na mesma posição



Fonte: O autor.

É importante cada posição da memória armazenar alguns *bits* extras para controle. O **bit válido** identifica se aquela posição na *cache* é válida ou não. Uma posição na *cache* não é válida se, por exemplo, diz respeito a um programa que já encerrou, ou se o computador foi recentemente ligado e ainda não tem nada mapeado. Temos também os **bits tag**, que armazenam os demais *bits* do endereço mapeado naquela posição da *cache*, servindo para identificar qual a posição exata da memória que está armazenada ali. Por exemplo, suponha que um programa acesse duas posições de memória “0010 0101” e “0001 0101”, nessa ordem. As duas serão mapeadas para a mesma posição na *cache*; no



momento em que o segundo acesso acontecer, será identificado que a posição 0101 da *cache* possui uma informação válida, porém, quando os *bits tag* forem verificados, será identificado o valor 0010 e não 0001. Portanto, será buscada a informação na memória principal e sobrescrito o valor do primeiro acesso pelo novo valor. Chamamos esse evento de *falha por conflito*.

As vantagens do mapeamento direto são sua simplicidade de implementação e velocidade de resolução do mapeamento. No entanto, a desvantagem está na presença das chamadas falhas por conflito, que são os erros decorrentes de uma posição livre na *cache* já estar ocupada por outro valor, mesmo existindo outras posições livres que teoricamente poderiam ser ocupadas, mantendo as duas informações na *cache*.

5.2 Mapeamento completamente associativo

No outro extremo do mapeamento direto, encontramos o mapeamento completamente associativo. Nessa modalidade de mapeamento, uma posição na memória pode ocupar qualquer posição na *cache*, eliminando a questão da falha por conflito. No entanto, faz-se necessário buscarmos por toda a *cache* a informação desejada, tornando a operação significativamente mais cara.

5.3 Mapeamento associativo por conjunto

Visando um meio termo entre as duas abordagens de mapeamento discutidas, temos o mapeamento de associatividade por conjunto, que se tornou a solução mais amplamente adotada. Nessa estratégia, de forma resumida, cada endereço pode ser mapeado não para uma única posição, mas para um conjunto delas. Por exemplo, cada posição poderia ser mapeada para, digamos, 4 posições distintas; no caso de uma falha por conflito ocorrer, teremos outras 3 posições de memória no *cache* para armazenar o dado, e no máximo apenas 4 comparações serão realizadas para decidir se um endereço da memória principal foi ou não mapeado na memória *cache*.

Nos mapeamentos associativo e associativo por conjunto, uma outra política deve ser adotada. Quando a memória *cache* enche e um novo bloco precisa ser armazenado, o sistema de memória deve escolher o bloco a ser removido para dar espaço ao novo bloco. No mapeamento direto, isso não existe porque cada bloco sempre fica na mesma posição.



5.4 Política de substituição

Outro conceito importante das memórias *cache* é a política de substituição. O que deve acontecer no mapeamento associativo completo ou por conjunto quando a *cache* fica cheia? A resposta é que algum bloco de memória deve ceder espaço para o novo bloco que está sendo acessado. Existem três estratégias principais para escolher qual bloco remover:

- **randômica**: nessa estratégia de substituição, um bloco da *cache* é removido de forma aleatória. Vantagens: fácil de implementar e rápido para decidir. Desvantagens: pode remover espaços frequentemente acessados na memória e se tornar pouco eficiente;
- **FIFO**: do inglês *First-In First-Out*, e no português também conhecido como “uma fila” – aquele que está a mais tempo na *cache* cede seu espaço;
- **LRU**: do inglês *Least-Recently Used*, no português se traduz como “o menos usado recentemente”. Quanto mais vezes uma posição na memória é acessada, maior sua “pontuação”, e quanto mais tempo passa sem ser acessada, menor ela é, removendo assim o bloco menos utilizado recentemente. Essa abordagem tira máximo proveito do princípio de localidade temporal.

FINALIZANDO

Nesta aula, iniciamos nossos estudos com uma introdução sobre a computação paralela, sua importância e a razão de ter se tornado padrão de arquitetura nos computadores modernos de uso geral. Discutimos a Lei de Moore e vimos a evolução dos computadores e limitações físicas encontradas nos *chips*. Tratamos da arquitetura Von Neumann, compreendendo os principais componentes de um sistema computacional. Além disso, estudamos a história por trás da evolução dos processadores e as dificuldades e gargalos encontrados ao longo das gerações desses *hardwares*.

Por fim, discutimos também as formas de mapeamento da *cache*, dispositivo responsável por desobstruir o acesso ao barramento principal.

Com esse estudo inicial, temos agora uma boa visão geral sobre os elementos que compõem os sistemas computacionais e suas finalidades.



REFERÊNCIAS

- BRESHEARS, C. **The Art of Concurrency**. Newton: O'Reilly, 2009.
- CHANDRA, R. et al. **Parallel programming in OpenMP**. Burlington: Morgan-Kaufmann, 2001.
- CHAPMAN, B.; JOST, G.; VAN DER PAS, R. **Using OpenMP: Portable Shared Memory Parallel Programming**. New York: MIT Press, 2008.
- DANTAS, M. **Computação distribuída de alto desempenho**. São Paulo: Axcel, 2005.
- DE ROSE, C. A. F.; NAVAUX, P. O. A. **Arquiteturas paralelas**. São Paulo: Bookman, 2005.
- KIRK, D. B.; HWO, W. W. **Programming Massively Parallel Processors: A Hands-on Approach**. Burlington: Morgan-Kaufmann, 2010.
- KUMAR, V.; KARYPIS, G.; GUPTA, A. **A Introduction to parallel computing**. Londres: Pearson, 2003.
- PACHECO, P. S. **An Introduction to Parallel Programming**. Burlington: Morgan Kaufmann, 2011.
- RAUBER, T.; RÜNGER, G. **Parallel programming: for multicore and cluster systems**. Berlin: Springer, 2010.
- STALLINGS, W. **Arquitetura e organização de computadores**. 10 ed. São Paulo, Pearson, 2017.